

Hotel Booking Ops V5

Developer Setup Guide — Getting the project running after cloning

This guide explains how to turn a fresh clone of the repository into a fully working Laravel application. It is written for new team members and assumes you are starting from a clean machine with the project just cloned from GitHub.

1. Project Overview

This is a Laravel 12 application using the following stack:

- **Backend:** PHP 8.2+ with Laravel 12, Livewire 3, Sanctum
- **Frontend:** Vite 7, Tailwind CSS 3, Alpine.js
- **Database:** MySQL (via XAMPP) or SQLite
- **Tooling:** Composer, npm, Pest (tests), Pint (formatting)

2. Why the Repository Looks “Incomplete”

After cloning you will **not** see folders like `vendor/`, `node_modules/`, or a `.env` file. This is intentional — these are listed in `.gitignore`. They are never committed because every developer regenerates them locally from the files that *are* committed (such as `composer.json`, `package.json`, and `.env.example`).

Missing item	How it is restored	Source of truth
<code>vendor/</code>	composer install	<code>composer.json</code> / <code>composer.lock</code>
<code>node_modules/</code>	npm install	<code>package.json</code> / <code>package-lock.json</code>
<code>public/build/</code>	npm run build	Vite + your source assets
<code>.env</code>	copy from <code>.env.example</code>	<code>.env.example</code>
Application key	php artisan key:generate	generated locally
<code>public/storage</code>	php artisan storage:link	symlink to <code>storage/app/public</code>
storage caches & logs	created by Laravel at runtime	n/a

Why it matters: Committing these would bloat the repo, cause constant merge conflicts, and leak secrets. Keeping them ignored is the correct, standard Laravel practice.

3. Prerequisites

Make sure the following are installed before you begin:

- **PHP 8.2 or higher** (XAMPP already provides this)
- **Composer** (PHP dependency manager)
- **Node.js 18+ and npm** (for compiling front-end assets)
- **MySQL** (bundled with XAMPP) or SQLite
- **Git**

4. Quick Start (Recommended)

The project ships with a **setup** script in **composer.json** that automates the whole process. From the project folder, run:

```
git clone <your-repo-url>
cd Hotel_Booking_Ops
composer setup
```

That single **composer setup** command performs all of the following in order:

```
composer install      # restore PHP dependencies (vendor/)
copy .env.example .env # create the environment file
php artisan key:generate # generate the APP_KEY
php artisan migrate   # build the database schema
npm install           # restore JS dependencies (node_modules/)
npm run build          # compile assets into public/build/
```

Before running: Configure your database first (see Section 6). The **migrate** step will fail if the database connection is not set up.

5. Manual Setup (Step by Step)

Prefer to run things one at a time, or need to debug a failed step? Run these commands in order from the project root:

1. Install PHP dependencies:

```
composer install
```

2. Create your environment file:

```
copy .env.example .env      # Windows
cp .env.example .env        # macOS / Linux
```

3. Generate the application encryption key:

```
php artisan key:generate
```

4. Configure the database in `.env`, then build the schema:

```
php artisan migrate  
# or, to also load demo/seed data:  
php artisan migrate --seed
```

5. Install and compile front-end assets:

```
npm install  
npm run build
```

6. Link the public storage folder (for uploaded images, etc.):

```
php artisan storage:link
```

6. Database Configuration

Open the `.env` file and set your database connection. For a typical XAMPP / MySQL setup, create an empty database in phpMyAdmin first, then set:

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=hotel_booking_ops  
DB_USERNAME=root  
DB_PASSWORD=
```

If you prefer `SQLite` (no server needed), set `DB_CONNECTION=sqlite` and create the database file:

```
type nul > database\database.sqlite # Windows  
touch database/database.sqlite      # macOS / Linux
```

7. Running the App Day to Day

A convenience `dev` script runs the PHP server, queue worker, and Vite dev server together:

```
composer dev
```

Or start pieces individually:

- `php artisan serve` — start the web server (`http://127.0.0.1:8000`)
- `npm run dev` — Vite dev server with hot reloading for assets
- `php artisan queue:listen` — process background jobs

8. Useful Commands Reference

Command	What it does
<code>composer setup</code>	Full one-shot setup of a fresh clone
<code>composer dev</code>	Run server + queue + Vite together
<code>composer test</code>	Run the Pest test suite
<code>php artisan migrate:fresh --seed</code>	Wipe & rebuild DB with seed data
<code>php artisan optimize:clear</code>	Clear all cached config/routes/views
<code>./vendor/bin/pint</code>	Auto-format PHP code to project style
<code>npm run build</code>	Recompile front-end assets for production

9. Troubleshooting

Symptom	Likely cause & fix
"No application encryption key has been specified"	Run <code>php artisan key:generate</code> .
SQLSTATE / database connection error	Check <code>.env</code> DB settings and that MySQL is running in XAMPP.
Vite manifest not found	Run <code>npm install</code> then <code>npm run build</code> .
Styles / images not loading	Run <code>php artisan storage:link</code> and rebuild assets.
Class not found after pulling changes	Run <code>composer install</code> and <code>composer dump-autoload</code> .

Tip: After pulling new changes from GitHub, it is good practice to run `composer install`, `npm install`, and `php artisan migrate` again in case dependencies or the schema changed.